

Microblog Platform

A reference demo plan for the Microblog platform — a multi-tenant microblog with real-time fan-out timelines, threaded conversations, media uploads, full-text search, and notifications.

Generated January 15, 2026 at 1:00 PM · demo-seed

Contents

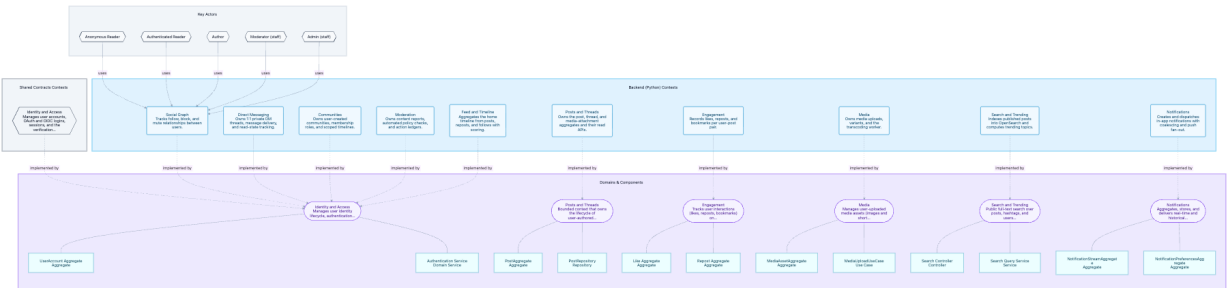
System Overview	Section 1
Architecture Layers	Section 2
Per-Layer Architecture Highlights	Section 3
Domain Deep Dive	Section 4
Key Workflows	Section 5
Architecture Decisions (ADRs)	Section 6
Glossary & Appendix	Section 7

System Overview

The Microblog Platform is a distributed multi-tenant microblog with real-time fan-out timelines, threaded conversations, media uploads, full-text search, and notifications, replacing a legacy single-server blog.

Key actors	Anonymous Reader, Authenticated Reader, Author, Moderator (staff), Admin (staff)
Constraints	Public timelines < 2s on 4G; fan-out p95 < 1s; DMs envelope-encrypted at rest
NFRs	p95 home feed < 1.5s warm cache; 99.9% read API availability; WCAG 2.1 AA; zero PII in logs

Figure 1 —



Blueprint — actors to contexts to components

The blueprint synthesises the system-wide shape from actors through the 11 bounded contexts down to the 24 components and 5 architectural layers (Backend, Backend Workers, Frontend, Shared Contracts, Infrastructure).

Figure 2 —

C4 context — how users reach the system

At the system context level, Readers, Authors, and Staff reach the Microblog over HTTPS. The platform validates OIDC and OAuth tokens against the Identity Provider, dispatches push notifications, and ships index events to a Search Indexer.

Figure 3 —

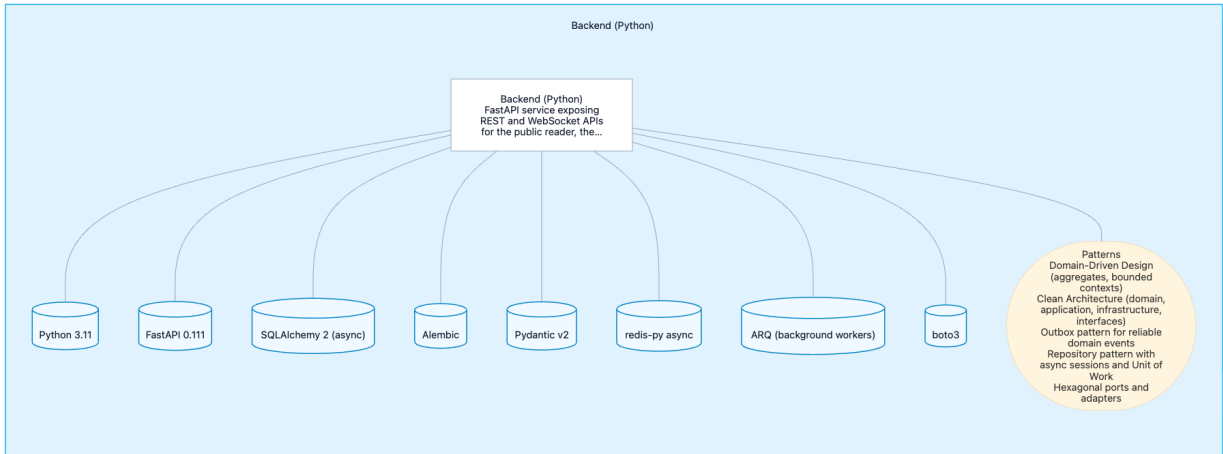
C4 container — runtime topology

The container diagram names the runtime topology: an Angular 17 SPA with PrimeNG talks to a Python 3.11 FastAPI API over HTTPS and WSS. ARQ workers consume Redis Streams; PostgreSQL 16, Redis 7, and S3 back the system.

Architecture Layers

Five layers compose the stack: a Python FastAPI backend, ARQ-based workers, an Angular SPA, shared Zod-rooted contracts, and a Terraform-managed Kubernetes runtime.

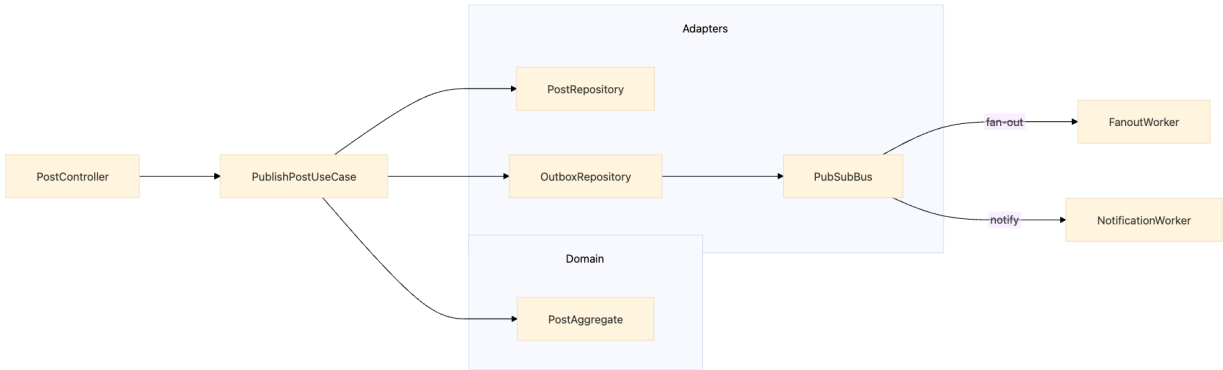
Figure 4 —



Backend — Python 3.11, FastAPI 0.111, SQLAlchemy 2

Name	Backend (Python)
Tech stack	Python 3.11, FastAPI 0.111, SQLAlchemy 2 (async), Alembic, Pydantic v2, redis-py async, boto3
Patterns	DDD aggregates, Clean Architecture, Outbox pattern, Repository + UoW, Hexagonal ports and adapters
Description	FastAPI service exposing REST and WebSocket APIs for publish, reply, engagement, and social-graph bounded contexts

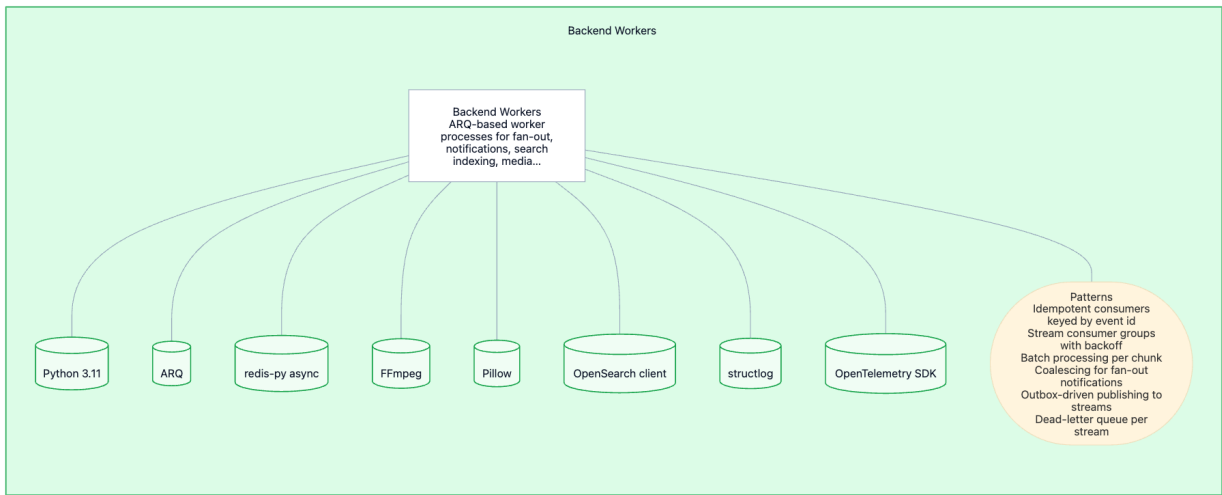
Figure 5 —



Layer overview — controller to use case to aggregate

The diagram shows the PostController routing through PublishPostUseCase into the PostAggregate, with OutboxRepository and PubSubBus writing fan-out and notification events to ARQ workers.

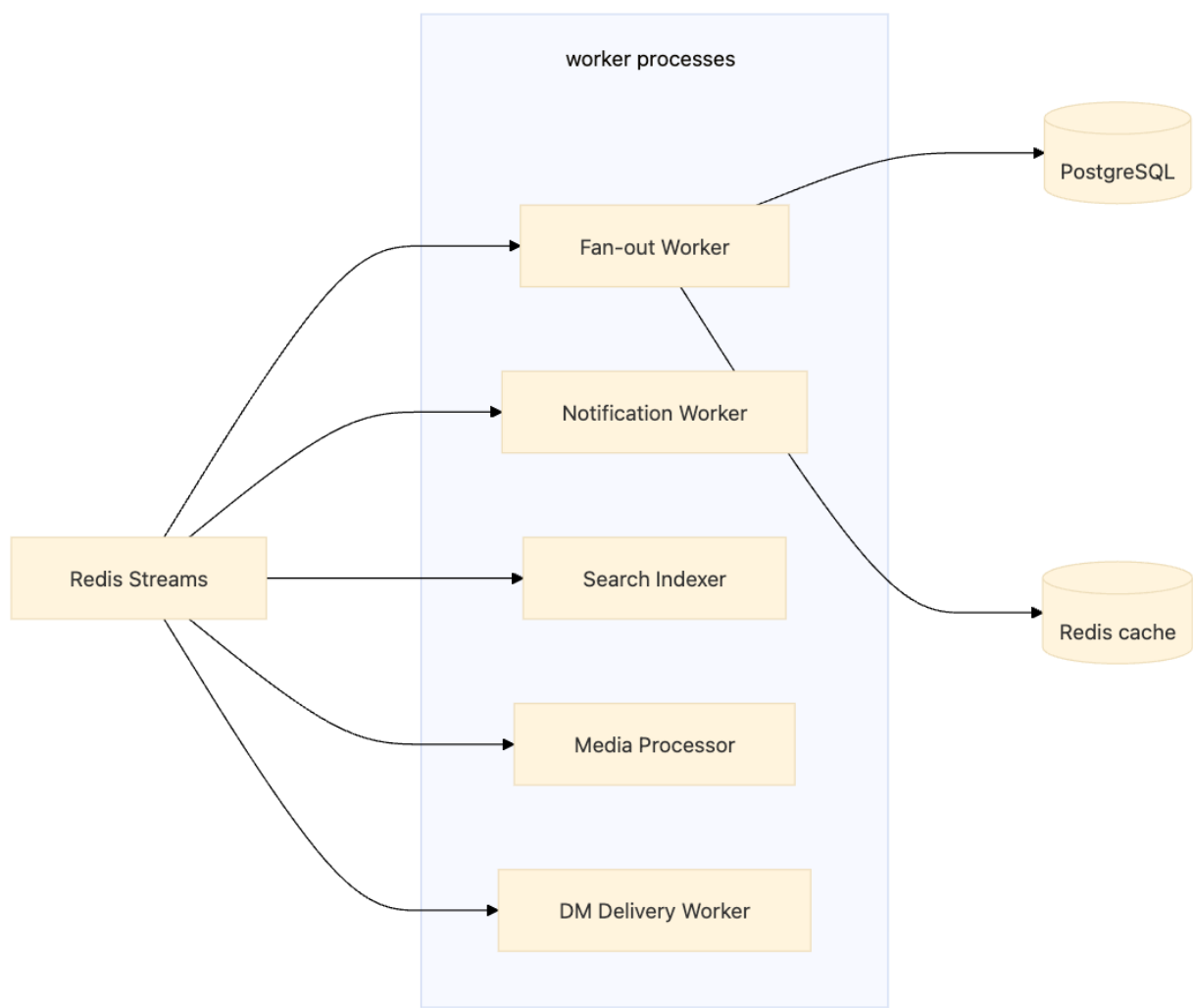
Figure 6 —



Backend Workers – Python 3.11, ARQ, Redis Streams

Name	Backend Workers
Tech stack	Python 3.11, ARQ, redis-py async, FFmpeg, Pillow, OpenSearch client, structlog, OpenTelemetry SDK
Patterns	Idempotent consumers, Stream consumer groups, Batch processing, Coalescing, Outbox-driven publishing, DLQ per stream
Description	ARQ worker processes for fan-out, notifications, search indexing, media transcoding, and DM delivery consuming Redis Streams

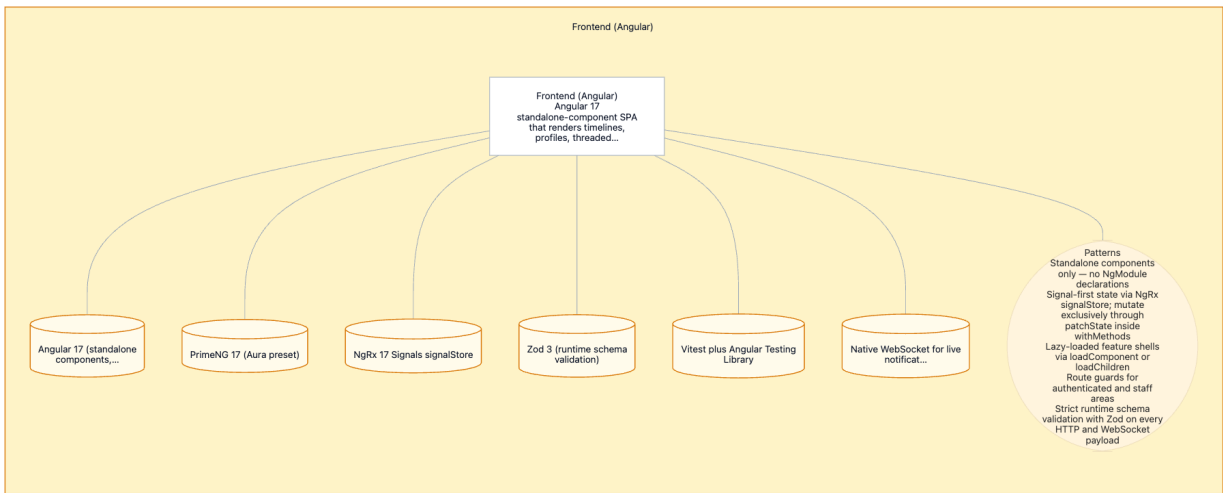
Figure 7 –



Layer overview – five worker kinds consuming Redis Streams

Five worker kinds (Fan-out, Notification, Indexer, Media, DM Delivery) read from Redis Streams and write to PostgreSQL, Redis cache, OpenSearch, or the push gateway, each idempotent on event id.

Figure 8 –

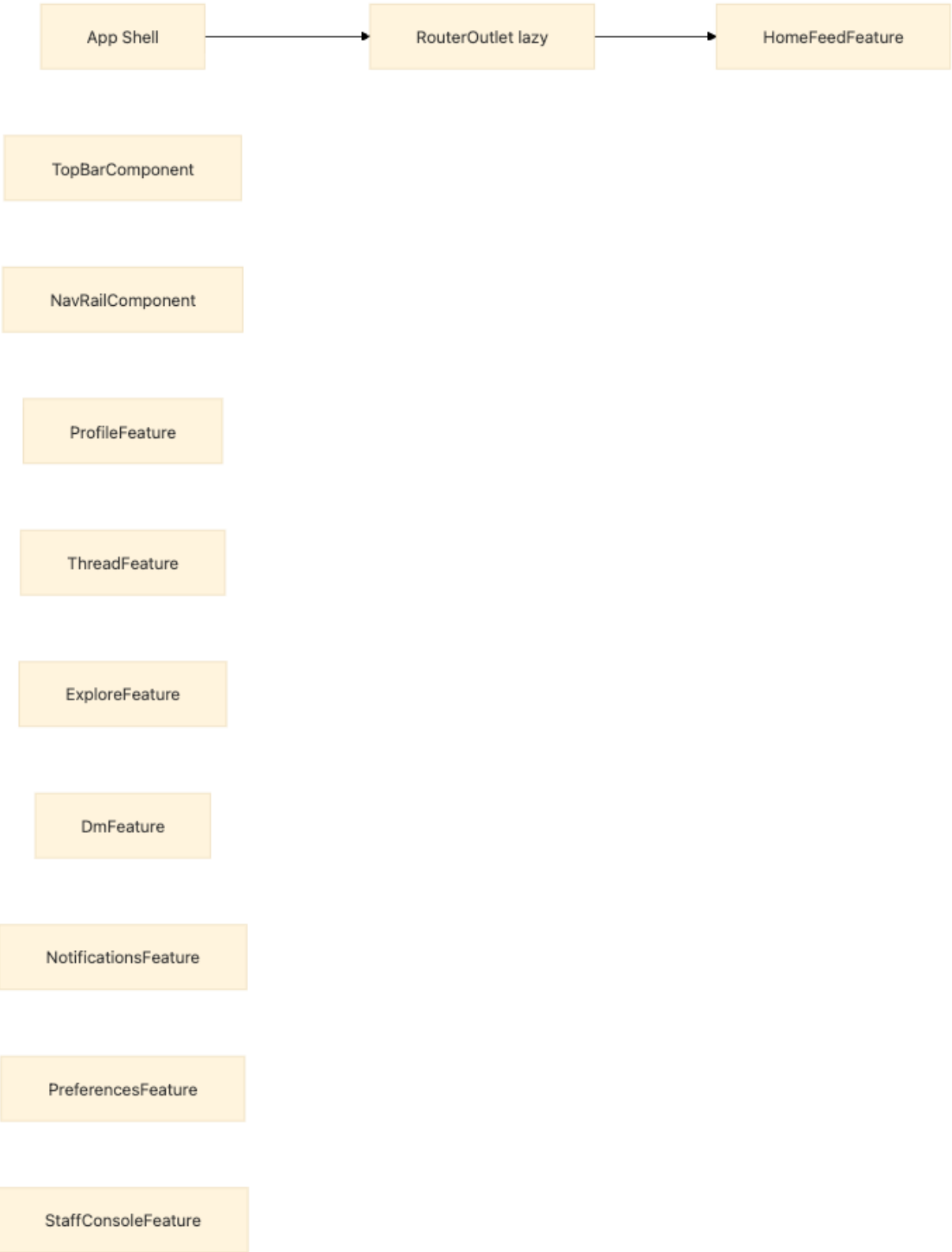


Frontend – Angular 17, NgRx 17, PrimeNG 17

Name	Frontend (Angular)
------	--------------------

Tech stack	Angular 17, PrimeNG 17, NgRx 17 signalStore, Zod 3, Native WebSocket, Vitest + Angular Testing Library
Patterns	Standalone components only, Signal-first state via NgRx signalStore, Lazy loadComponent, Route guards, Zod runtime validation
Description	Angular 17 standalone-component SPA with lazy-loaded features; Zod-validated HTTP/WSS payloads into NgRx signalStores

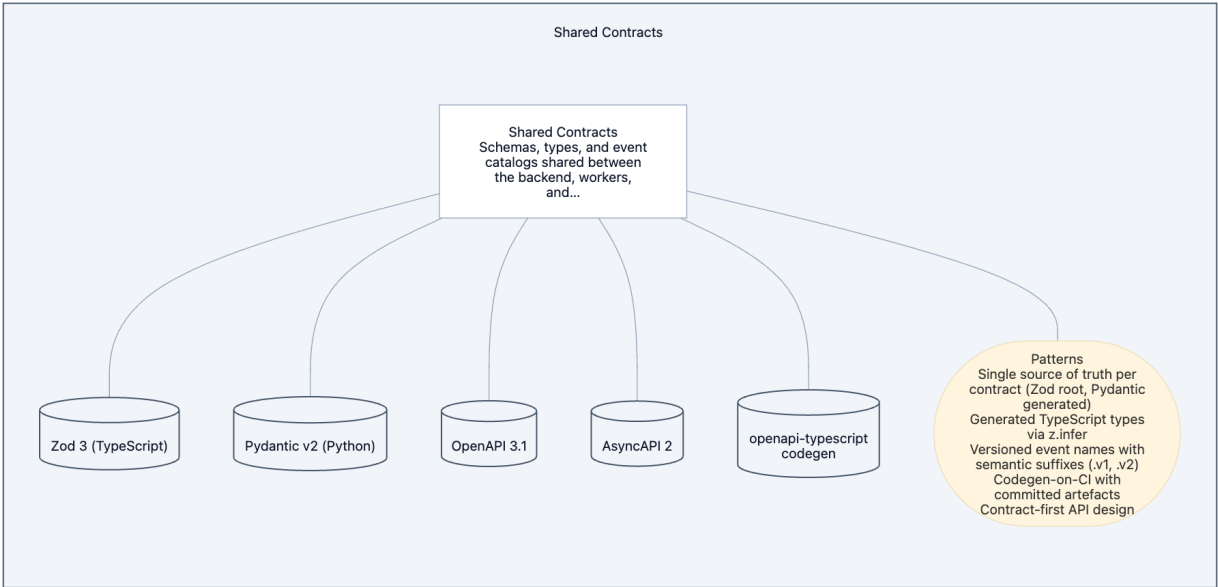
Figure 9 —



Layer overview – lazy features and signal stores

The diagram shows the App Shell routing to lazy-loaded features (Home, Profile, Thread, Explore, DMs, Notifications, Preferences, Staff), each consuming shared stores and the HttpClient.

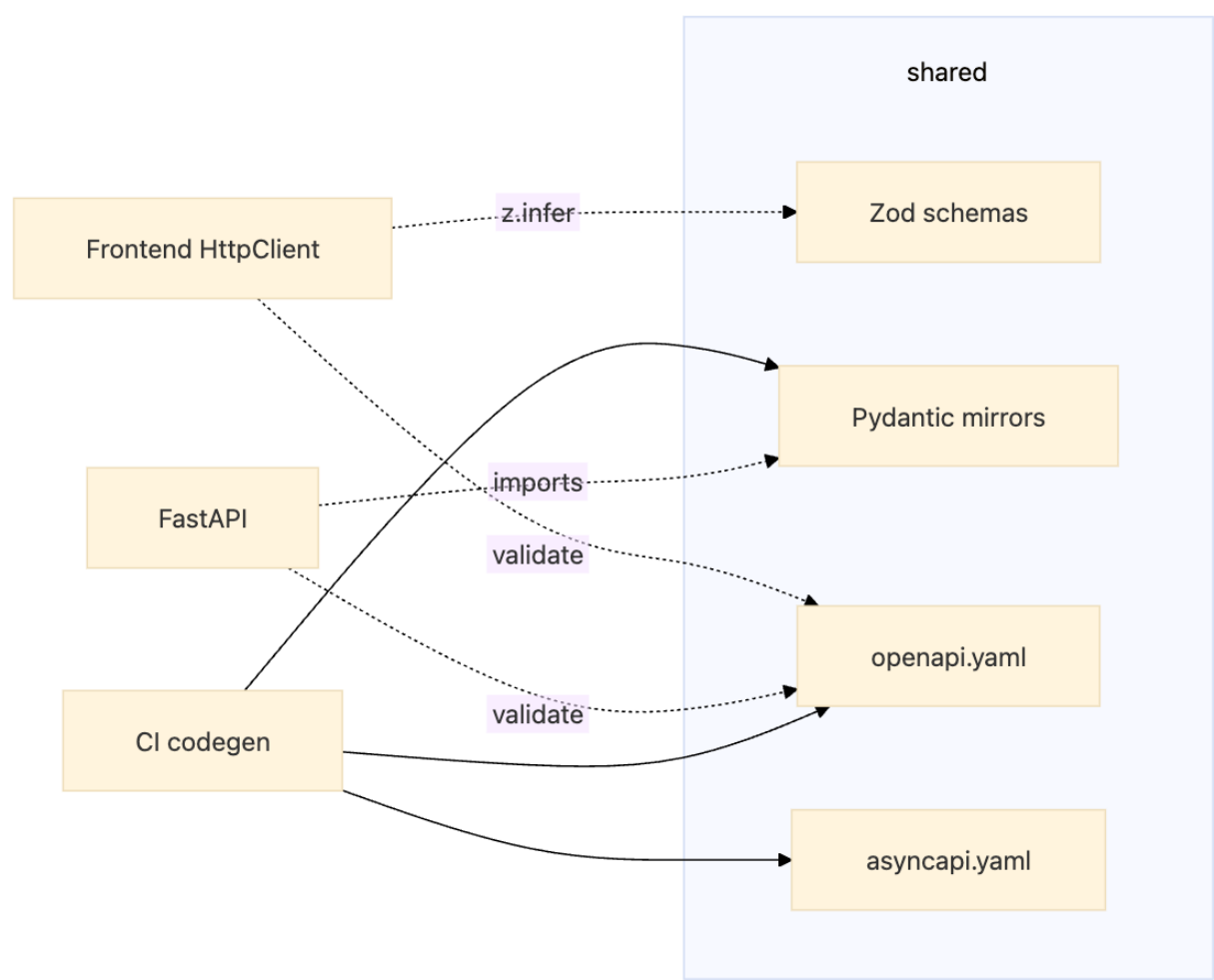
Figure 10 –



Shared Contracts – Zod 3, Pydantic v2, OpenAPI 3.1

Name	Shared Contracts
Tech stack	Zod 3, Pydantic v2, OpenAPI 3.1, AsyncAPI 2, openapi-typescript codegen
Patterns	Single source of truth (Zod root + Pydantic mirror), z.infer generation, Versioned event names, Codegen-on-CI, Contract-first
Description	Zod-rooted schemas mirrored to Pydantic; TypeScript types via z.infer; OpenAPI 3.1 and AsyncAPI 2 artefacts committed to repo

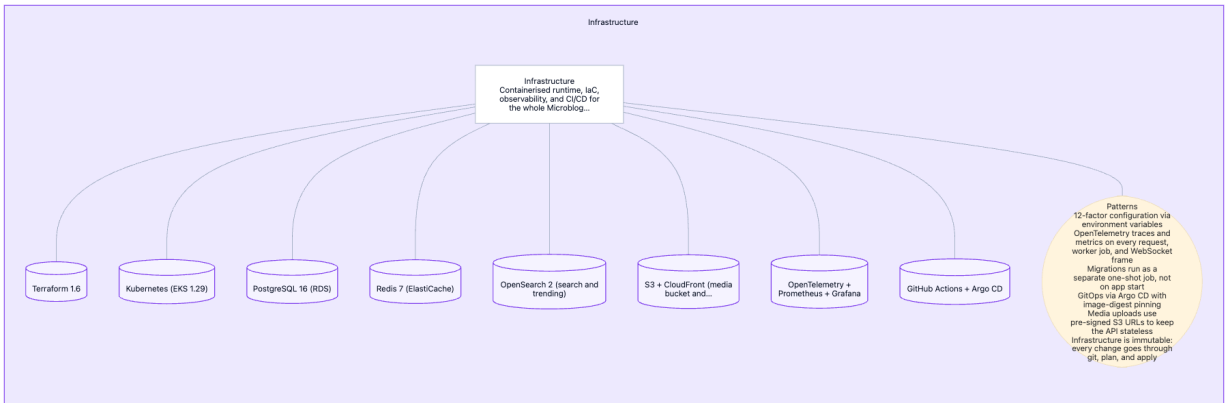
Figure 11 –



Layer overview – Zod root fans out to Pydantic, OpenAPI, AsyncAPI

The diagram shows Zod schemas as the single root, fanning out to Pydantic mirrors, openapi.yaml and asyncapi.yaml via CI codegen, with FastAPI importing Pydantic and the SPA importing z.infer types.

Figure 12 –

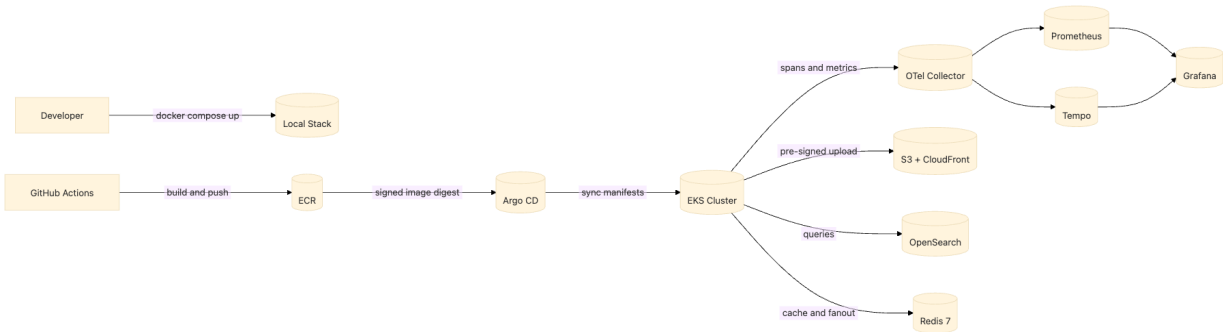


Infrastructure – Terraform 1.6, Kubernetes 1.29, Argo CD

Name	Infrastructure
Tech stack	Terraform 1.6, Kubernetes (EKS 1.29), PostgreSQL 16 RDS, Redis 7 ElastiCache, OpenSearch 2, S3+CloudFront, OTel+Prom+Grafana

Patterns	12-factor config, OTel traces/metrics, Migrations as separate one-shot job, GitOps via Argo CD, Image-digest pinning
Description	Containerised runtime on EKS managed by Argo CD; OpenTelemetry, Prometheus, Grafana for observability; presigned S3 uploads

Figure 13 —



Layer overview — CI to ECR to Argo CD to EKS

The diagram traces CI building images to ECR, Argo CD syncing manifests to EKS, the cluster emitting spans to OTEL Collector, Prometheus, and Tempo, with sidecars issuing pre-signed S3 uploads.

Per-Layer Architecture Highlights

The five layers carry distinct technical contexts, constitution rules, and tracked complexity. The matrices below compare storage, runtime, performance, constraints, and scale across all layers.

	Backend	Workers	Frontend	Shared	Infra
Storage	PostgreSQL 16 + Redis 7 + S3 + OpenSearch 2	Redis Streams in; PostgreSQL/ S3/OpenSearch out	Browser memory only; HttpOnly session cookies	Zod sources + codegen artefacts; no runtime DB	GH Actions cache + ECR digest-tagged + S3 media
Target platform	Linux server, Python 3.11, Docker	Linux containers amd64 + arm64	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+	Node 20 + Python 3.11 codegen on CI	Kubernetes 1.29 (EKS), amd64 + arm64
Performance	p95 GET home < 200ms; p95 POST < 400ms; 50k posts/min	10k events/min/replica; p95 < 1s	TTI < 3s cold; LCP < 2s on 4G; first feed < 1.5s	Codegen < 30s on CI; Zod < 1ms per request	CI < 12min; deploy < 5min; p95 feed < 200ms
Constraints	Zero framework imports in domain; outbox-only events; auth in app layer	Idempotent on event id; exp backoff up to 5; DLQ per stream	Signals only; patchState-only mutations; Zod on every payload; no NgModules	No hand-written TS types; .vN+1 suffix on breaking events; OAS committed	Digest-pinned images; migrations as one-shot jobs; runbook-linked alerts
Scale	1M MAU; 50k posts/min peak; 200 RPS read; 50 RPS write	1k jobs/replica across 6 kinds × 3 replicas	1M MAU; 50k posts/min; ~100k WSS connections	~80 Zod schemas; ~30 AsyncAPI events; 12 contexts	3 envs; ~80 services; ~20TB PG; ~50TB S3; 10M DAU

Spec-kit technicalContext across all five layers

- Backend – zero framework imports; strict Pydantic; ≥ 2 unit + 1 integration per use case
- Workers – idempotent on event id; OTel span per job; /healthz; DLQ after 5 retries
- Frontend – strict TS; standalone components; patchState-only; Zod on every payload; WCAG 2.1 AA
- Shared – Zod source of truth; codegen drift fails CI; .vN+1 suffix on breaking events
- Infra – digest-pinned images; migrations as jobs; every alert links to a runbook; OTel traces

	Rationale	Layers
Outbox table (Backend)	At-least-once delivery to ARQ without dual-write inconsistencies	Backend
Redis Streams consumer groups (Workers)	Native backpressure, replay, and per-stream DLQ	Backend Workers

NgRx signalStore (Frontend)	Audit boundary between reads and writes via patchState	Frontend
Zod validation on payload (Frontend)	Catches upstream contract drift at the SPA/backend seam	Frontend
Schema parity (Shared)	Independent evolution without silent contract drift	Shared
OTel+Prom+Grafana (Infra)	Single pane of glass across web, API, workers, and WSS	Infra

Complexity-tracking violations per layer (selected items)

Domain Deep Dive

Six backend contexts own the aggregates and domain events that drive fan-out across the platform: identity, posts, engagement, media, search, and notifications.

- Register accounts with unique username, email, and hashed credentials
- Authenticate users via password verification; issue short-lived tokens
- Maintain profile attributes: display name, bio, avatar URL
- Handle verification, password reset, and account deactivation flows

UserAccount	Username <code>^[a-z0-9_]{3,20}\$</code> ; email unique; password stored only as hash
--------------------	---

- user-registered — Register cmd → social-graph, notifications, search
- user-authenticated — AuthenticationService.authenticate → feed, social-graph, notifications
- profile-updated — UpdateProfile cmd → search, feed
- Create, edit, delete original posts up to 280 characters
- Author threaded replies and maintain parent-child structure
- Enforce authorship, immutability, and soft-delete invariants
- Attach media references (image/video) to posts

Post	Body 1–280 chars; only author may edit/delete; edits blocked after 24h
Thread	Root is top-level; reply count = non-deleted children; cycles rejected

- post-created — CreatePost|ReplyToPost → engagement, feed, notifications
- post-edited — EditPost cmd → feed, search
- post-deleted — DeletePost cmd → engagement, feed, notifications
- Record and revoke likes with single-like-per-user enforcement
- Record simple and quote reposts with original authorship
- Manage user-private bookmarks and bookmark collections
- Expose denormalized like/repost counters for feed assembly

Like	One per user-post; refs existing post and user; unlike only when active
Repost	No self-reposts; quote 1–280 chars; preserves original author and id
Bookmark	One per user-post; visible only to owner; collection name ≤ 32 chars

- post-liked — Like.LikePost → feed, notifications, posts
- post-reposted — Repost.RepostPost|QuoteRepost → feed, notifications, posts
- post-bookmarked — Bookmark.BookmarkPost → feed
- Coordinate resumable uploads of images and short videos
- Validate media metadata (type, size, dimensions) against rules
- Trigger and reconcile async processing: transcode, thumbnails, scans
- Issue short-lived CDN URLs and manage per-asset derivatives

MediaAsset	Size ≤ per-type limit (5MB img, 40MB vid); MIME on whitelist; no Failed/Deleted reversal
-------------------	--

MediaAttachment	≤ N per post; requires Ready MediaAsset; scoped to single post
- media-ready — MediaAsset.MarkReady ☐ posts, feed, moderation - media-processing-failed — MediaAsset.MarkFailed ☐ posts, moderation - media-deleted — MediaAsset.Delete ☐ posts, feed, moderation - Provide full-text search across posts, hashtags, and user profiles - Compute trending topics from rolling window with periodic recompute - Maintain synchronized search index from upstream domain events - Apply ranking, pagination, and content-safety filters to queries	
SearchIndex	Each doc refs source id; removed sources tombstoned; analyzer version pinned
TrendingTopic	Window is positive multiple of interval; ranked by descending score; pinned persists
- search-index-updated — IndexDocument RemoveDocument ☐ posts, engagement, feed - trending-snapshot-computed — RecomputeTrends ☐ feed, notifications - trending-topic-promoted — RecomputeTrends ☐ feed, notifications - Record notifications from Posts, Engagement, Social Graph, Messaging, Moderation - Aggregate to collapse duplicates (e.g. many likes ☐ one digest) - Deliver via in-app inbox, WSS push, mobile push, and email - Manage user preferences, quiet hours, and per-type/per-channel muting	
NotificationStream	Owner userId required; UNREAD☐ READ☐DISMISSED only; MarkAllAsRead scoped to stream
NotificationPreferences	At least one channel enabled; quiet hours valid range; mute txn-locked
NotificationDelivery	Requires NotificationId; retries ≤ 5; terminal states reject further transitions
- notification-recorded — NotificationStream.RecordNotification ☐ dispatcher, feed-projector - notification-read — NotificationStream.MarkAsRead ☐ repository, unread-counter-projector - notification-preferences-updated — NotificationPreferences.SetChannelPreference ☐ dispatcher	

Key Workflows

Six end-to-end workflows span the backend, workers, and frontend: publish-and-fanout, engage-and-notify, upload-and-process, search-and-trend, report-and-enforce, and join-and-participate-community.

1. Author opens composer and clicks Post.
2. POST /api/posts persists post + outbox event in one tx
3. PubSub bus publishes post.published.v1 to fan-out + search streams
4. Fan-out dispatcher loads followers; writes timeline rows in 500-row batches
5. Timeline cache prepends new post id; trims to 800 entries
6. SPA home feed shows the new post on refresh or pull-to-refresh

A post reaches every follower's home feed within 1 second, so conversations feel synchronous across the network.

- posts
- social-graph
- search

1. Reader likes a post from home feed or thread view.
2. POST /api/likes persists like and writes engagement.recorded.v1
3. Reply or repost persists new post; emits reply.created.v1 or repost.created.v1
4. Notification worker detects events; creates Notification aggregate
5. Fan-out scheduler coalesces notifications; dispatches WSS push
6. Author sees the notification within 2 seconds

An author learns about likes, replies, and mentions in under 2 seconds, keeping conversations live.

- engagement
- notifications
- posts

1. Author attaches image or short video in the composer.
2. Backend issues presigned URL; records upload as pending
3. Client uploads to presigned URL and calls markUploaded
4. Media worker transcodes into thumbnail, medium, and full variants
5. Variants uploaded to object store; CDN URLs returned
6. Post publish flow attaches variants; emits post.published.v1

A composer attachment finishes transcoding in seconds and the post ships with CDN-ready media variants.

- media
- posts

1. Fan-out emits post.published.v1 and post.deleted.v1 to indexer stream
2. Indexing queue buffers posts up to 500 items or 2 seconds
3. Queue flushes bulk request to OpenSearch; retries with backoff on failure
4. Trending calculator applies half-life decay to engagements per hashtag
5. Explore page serves top-N trending hashtags from 60s snapshot

Search returns in under 100ms p95 and trending refreshes every 60 seconds with half-life decay.

- search
- posts

1. Reader clicks Report on a post; submits a form with a reason
2. POST /api/reports persists report; emits report.created.v1
3. Triage service assigns report to least-loaded moderator with capacity
4. Staff reviews report; applies enforcement via BlockList
5. Backend persists block; emits graph.blocked.v1 or post.hidden.v1
6. Reported post is hidden from reporter's timelines within 5s

A reported post is hidden from the reporter's timelines within 5 seconds, while staff triages via the console.

- social-graph
- posts
- moderation

1. Visitor browses Communities directory; opens community page
2. User clicks Join; membership persisted; community.joined.v1 emitted
3. Community composer validates body + media against community rules
4. POST /api/posts with communityId persists post scoped to community
5. Fan-out writes post to community-member timelines only
6. Community moderators can pin or remove posts via moderation tool

A community post reaches members within seconds, scoped away from the global home feed.

- communities
- posts
- moderation

Architecture Decisions (ADRs)

Six ADRs capture the accepted architectural commitments: fan-out on write for timelines, PostgreSQL plus Redis as the primary store, OpenSearch for search and trending, S3 plus CDN for media, NgRx signalStore for frontend state, and Zod-rooted shared schemas.

ADR-0001 — Fan-out on write for home timelines. Home timelines must render in under 200ms p95 from a warm cache; fan-out on read would force a join over follow edges on every read.

Decision: use fan-out on write via a Redis stream and a dedicated worker that materialises per-user timeline rows and caches.

- Hot authors (verified accounts) require sharded workers to keep up.
- Unfollow requires dematerialisation via `graph.unfollowed.v1`.
- Read latency drops from multi-second joins to sub-50ms cache reads.
- Community posts fan out only to community members.

ADR-0002 — PostgreSQL plus Redis for the primary store. We need strong consistency for posts, follows, and engagement with sub-millisecond read latency for hot timelines.

Decision: use PostgreSQL 16 as the system of record and Redis 7 for the hot-path timeline cache and Redis Streams.

- Postgres handles transactional outbox writes for fan-out reliability.
- Redis is treated as a cache; a cold cache falls back to Postgres reads.
- Redis failover can lose up to 5 seconds of cache writes; reads recover from Postgres.

ADR-0003 — OpenSearch for full-text and trending. Typeahead and full-text search must return results in under 100ms p95 across tens of millions of posts.

Decision: index published posts into OpenSearch with a dedicated indexer worker and a buffered bulk request strategy.

- A deleted post is removed from the index within 5 seconds.
- Trending snapshots are recomputed from the index with a 60-second TTL.
- Index failures fall back to a Postgres full-text search query path.

ADR-0004 — S3-compatible object storage plus CDN for media. Media uploads include images up to 50MB and videos up to 500MB and must be served with low latency worldwide.

Decision: store raw and transcoded media in S3-compatible object storage and serve variants through a CDN.

- Presigned URLs keep uploads off the API server.
- Variant generation is idempotent on upload id.
- CDN cache invalidation is required when a media variant is moderated.

ADR-0005 — NgRx signalStore for frontend state. We need fast, signal-based state management with a clear audit boundary between reads and writes.

Decision: standalone Angular components only, NgRx signalStore for feature state, mutate only via `patchState` inside `withMethods`.

- Components cannot mutate state directly; they call store methods.
- Schema validation runs at the API service boundary before reaching stores.

- Smaller bundles and clearer feature ownership.

ADR-0006 – Zod schemas shared between frontend and backend. Type drift between the Angular SPA and the FastAPI backend has caused production schema_drift errors.

Decision: write every API contract once in Zod (frontend) and mirror it in Pydantic (backend); generate TypeScript types via `z.infer` and OpenAPI from Zod.

- No hand-written TypeScript request or response types.
- Schema changes require a codegen run on CI; CI fails on uncommitted generated artefacts.
- Breaking changes require a new `.vN` suffix on the event name.

Glossary & Appendix

Consolidated glossary of the ubiquitous language used across the 11 bounded contexts: identity, social graph, posts and threads, engagement, media, search, notifications, messaging, communities, moderation, and feed.

UserAccount	A registered user with a unique handle, display name, and verification state.
Session	A server-side record of an authenticated client bound to a refresh token id.
VerificationBadge	A per-user trust indicator granted by the verification-badge domain service.
FollowEdge	A directed edge from follower to followee, with a creation timestamp.
BlockList	A per-user list of blocked accounts; blocks suppress interactions both ways.
MuteList	A per-user list of muted accounts; mutes hide posts but preserve follows.
Post	A short post with up to 280 characters and a lifecycle status.
Thread	An ordered self-reply chain anchored on a root post.
MediaAttachment	A reference from a post to one or more media variants owned by the media context.
Like	A user-post pair indicating a positive engagement.
Repost	A user-post pair indicating a repost; plain or quote.
Bookmark	A private per-user pointer to a post, ordered by recency.
MediaUpload	A single upload tracked from request through ready variants.
MediaVariant	A single transcoded rendition (thumbnail, medium, full, HLS).
MediaProcessor	A service that turns a raw upload into ready variants.
SearchIndex	The OpenSearch index of published posts with typeahead and full-text search.
TrendingItem	A ranked hashtag or topic over a rolling 24-hour window with half-life decay.
IndexingQueue	An in-memory buffer that batches index events to OpenSearch.
Notification	A per-user record describing a like, repost, mention, follow, or DM event.
Mention	A handle reference extracted from a post or DM body for notification purposes.
FanoutScheduler	A coalescing dispatcher that decides when to push a notification.
DMConversation	A 1:1 conversation between two users with ordered messages.
DMMessage	A private text or media message with send and read state.
DeliveryReceipt	A per-recipient pointer to the last delivered message.
Community	A named group with members, posts, and moderators.

CommunityMembership	A user-to-community edge with owner, mod, or member role.
CommunityPost	A post whose visibility is scoped to a single community.
ContentReport	A user-submitted report against a post, DM, or account.
ModerationAction	A logged decision by an automated rule or moderator.
PolicyRule	A rule that emits ModerationActions when matched.
HomeTimeline	A materialized feed of recent posts from followed users.
FeedItem	A scored timeline entry: post, repost, or reply.
TimelineFanout	The merge of post events into follower timelines.